

Topics in Error-Correcting Codes

Notes by Yael Kalai

MIT - 6.S953

Lecture 8 Computationally Efficient Interactive Coding (October 5, 2026)

Warning: This document is a rough draft, so it may contain bugs. Please feel free to email me with corrections.

Recap

In the last three lectures we defined tree codes, showed how to use them for interactive coding, and proved that good tree codes exist. Recall that a tree code of rate R and relative bit distance δ gives a deterministic simulation with constant communication overhead that tolerates a $\delta/96$ fraction of bit corruptions [3].

However, this simulation is not computationally efficient. The tree-code construction from last lecture is a randomized construction that produces a good tree-code only with constant probability, and does not give an efficient way to check if the resulting construction is indeed a good tree code. It also does not give a way to efficiently decode in the presence of noise. One can decode and check if the tree code is good in time $2^{O(N)}$, where N is the depth of the tree.

In this lecture we show how to obtain a *computationally efficient* interactive coding scheme.

One approach can be to construct a tree-code that is efficiently encodable and efficiently decodable in the presence of error. Then we could use this in Schulman's interactive coding construction [4] to get a good interactive coding scheme. Constructing such a tree-code is a great question that is still open.

In this lecture we will see how to obtain a without solving either problem. We prove the following theorem.

Theorem 1. *There exist constants $\rho > 0$ and $C > 0$ and a randomized transformation which takes any deterministic binary protocol Π communicating k bits into a protocol communicating at most Ck bits, such that both parties recover the original transcript with probability $1 - 2^{-\Omega(k)}$, even if an adversary corrupts a ρ fraction of the transmitted bits. The transformation has polynomial computational overhead.*

The probability is over fresh randomness generated by the parties during the simulation. The adversary may be computationally unbounded, know both inputs, and choose corruptions adaptively. It may learn a random seed once it is generated, but it cannot know future random coins.

Remark. We note that our interactive coding scheme is randomized (even if the underlying protocol is deterministic). We have deterministic *non-uniform* good inter-

active coding schemes [2], but constructing good interactive coding schemes that are deterministic and remains an open problem.

We use the block-and-hash approach from [1]; see also [2].

Recall: Recall that the linear tree code construction from last lecture has a description of length $O(D)$ for depth D . We thus can enumerate all its coefficient sequences and check the distance condition on all inputs that begin with 1. Both the enumeration and the verification take $2^{O(D)}$ time. We can also decode a received string by enumerating all input histories, in time $2^{O(D)}$. Consequently, Schulman's interactive coding scheme satisfies the following theorem.

Theorem 2. *There exist constants $a > 1$ and $\eta > 0$ with the following property. Any binary protocol communicating D bits has a deterministic simulation communicating $L = aD$ bits, which recovers its transcript whenever at most ηL transmitted bits are corrupted. The tree code needed for this simulation can be constructed, and the simulation can be executed, with $2^{O(D)}$ computational overhead.*

Initial idea: Run Schulman's interactive coding scheme on chunks of length $O(\log k)$. Then the resulting protocol has only polynomial overhead! However, this only offers local decoding. The adversary can corrupt an entire chunk is corrupted, and there is no mechanism to later detect or correct this. Thus, protecting each chunk is not enough. We also need to check the *entire tentative transcript*, and to undo earlier progress when a check fails.

Main idea: In between these short simulations, use hashing to check whether the parties agree on their progress. If they do not, they backtrack or stay.

In what follows we assume that the underlying protocol has communication complexity k , and that the parties alternate sending one bit each.

Comparing Transcripts Using Hashing

We first describe the hash function that we will use. Choose a finite field $\mathbb{F}_Q$ of characteristic 2, where Q is a power of 2 satisfying $Q = \text{poly}(k)$. For $s \in \mathbb{F}_Q$ and $u = (u_1, \dots, u_\ell) \in \{0, 1\}^\ell$, where $\ell \leq k$, define

$$h_s(u) = \sum_{j=1}^{\ell} u_j s^{j-1} \in \mathbb{F}_Q. \quad (1)$$

The hash of the empty string is 0. We will only compare strings of the same length; their lengths are communicated separately.

Claim 1. For any two distinct strings u, v of the same length at most k ,

$$\Pr_{s \leftarrow \mathbb{F}_Q} [h_s(u) = h_s(v)] \leq \frac{k}{Q}$$

Proof. The difference $h_s(u) - h_s(v) = \sum_{i=0}^k (u_i - v_i)s^i$ is a nonzero polynomial in s of degree at most k . Thus, it has at most k roots in $\mathbb{F}_Q$, and hence the probability of s being a root is at most $\frac{k}{Q}$. $\square$

The fact that $Q = \text{poly}(k)$ implies that both the seed s and the value $h_s(u)$ have $O(\log k)$ bits, and the hash can be evaluated in polynomial time.

Why must the seed be fresh? The claim applies to two strings fixed *before* the seed is sampled. It need not hold if the strings are chosen after seeing the seed. At the end of every block we therefore freeze the tentative transcripts, sample uniformly random seeds, and send the hashes. We do not assume that the seeds remain secret.

The Construction

As in previous lectures, we assume that Π alternates between Alice and Bob, starting with Alice, and that its length $k \geq 1$ is known. Extend Π to a protocol consisting of $2k$ bits, where after its first k bits, the parties send dummy 0 messages. Set

$$b = \lceil \log_2 k \rceil, \quad N = 8 \cdot \frac{k}{b}, \quad (2)$$

where b is the block length, and we assume that it is even, so every block begins with the same designated speaker. The simulation consists of N blocks.

Each party $P \in \{A, B\}$ maintains a tentative transcript v_P whose length is a multiple of b . We think of v_P as a vertex in the original protocol tree. Initially, $v_A = v_B = \emptyset$.

A local consistency check. Call a string v *locally consistent with party P* if every bit in v sent by P agrees with P 's next-message function. Party P can check this without knowing the input to the other party. Note that a prefix of a locally consistent string remains locally consistent. We require every accepted tentative transcript to pass this check. This will ensure that any common prefix of the two tentative transcripts is a correct prefix of the original execution.

The Blocks

In what follows we describe the i 'th block in the noiseless setting. We then apply an interactive coding to this block. Since each block consists of $O(\log k)$ bits, we can afford the $2^{O(\log k)} = \text{poly}(k)$ blowup in computation.

At the beginning of the block, party P freezes v_P , sets $i_P = |v_P|/b$, samples an independent seed $s_P \leftarrow \mathbb{F}_Q$, and forms the block header

$$H_P = (i_P, s_P, h_{s_P}(v_P)).$$

Importantly, we assume that each header has the same fixed length $q = \theta(\log k)$.

Consider the following noiseless protocol, whose local inputs include the frozen transcripts and headers:

1. The parties exchange H_A and H_B .
2. They execute the next block of the underlying protocol, exchanging b bits. Party P uses the next-message function of the extended protocol Π , starting from its own frozen transcript v_P , appending the bits exchanged in this step.

Note that this protocol communicates $O(\log k)$ bits. It is well defined even when $v_A \neq v_B$: each party uses its own starting prefix, and the even block length ensures that the two parties agree on who sends each bit. When $v_A = v_B$ is a correct prefix, the last b bits are precisely its correct continuation.

The parties execute this *entire* block, including the headers and the b -bit continuation, using an interactive coding scheme that satisfies the guarantees of Theorem 2. For example, they can use Schulman's interactive coding scheme. Every encoded block has the same length

$$L = \theta(\log k) = \theta(b). \quad (3)$$

Updating the Tentative Transcript

Let P be one party and let Q be the other party. Party P parses its decoded block as two headers and a candidate continuation $u \in \{0,1\}^b$.

First, if parsing fails then P stays at v_P . Also, if P 's own header is not exactly H_P , then stay at v_P . Otherwise write the decoded other header as $(\hat{i}_Q, \hat{s}_Q, \hat{o}_Q)$ and proceed as follows:

1. If $i_P > \hat{i}_Q$, delete the last block of v_P .
2. If $i_P < \hat{i}_Q$, leave v_P unchanged.
3. If $i_P = \hat{i}_Q$ but $h_{\hat{s}_Q}(v_P) \neq \hat{o}_Q$, delete the last block of v_P .
4. If $i_P = \hat{i}_Q$ but $v_P \| u$ is not locally consistent with party P then stay at v_P .
5. Otherwise, append u to obtain $v_P \| u$.

Deleting a block from the empty string means staying at the empty string. Thus, in every round, each party either appends one block, deletes one block, or stays. In particular, no decoded header can make a party jump to an arbitrary depth or replace its entire transcript.

Efficiency. After N blocks, each party outputs the first k bits of its tentative transcript, or outputs failure if it has fewer than k bits. Note that the communication complexity is

$$N \cdot O(b) = 8 \cdot \frac{k}{b} \cdot O(b) = O(k),$$

as desired. Moreover the computational efficiency of the scheme follows from the efficiency of each block, together with the fact that updating the tentative transcript can be done efficiently.

Correctness of the Simulation

Fix the parties' inputs and an adaptive adversary satisfying the global error bound. Let π^* be the correct infinite transcript of the extended protocol. Write v_P^t for P 's tentative transcript after round t , and let w_t be the longest common prefix of v_A^t and v_B^t .

Claim 2. For every t , the string v_P^t is locally consistent with P . Consequently, w_t is a prefix of π^* .

Proof. Local consistency holds initially. It is preserved by staying or deleting a suffix, and it is explicitly checked before appending a block.

To prove the second assertion, suppose a bit of w_t is the first one that disagrees with π^* . The party that owns this position has the correct preceding prefix. Local consistency of its tentative transcript forces it to put the correct bit at this position, a contradiction. $\square$

We measure progress in blocks. Define

$$\ell_P^t = \frac{|v_P^t|}{b}, \quad g_t = \left\lfloor \frac{|w_t|}{b} \right\rfloor.$$

By Claim 2, it suffices to show that $g_N \geq k/b$. We prove that this is indeed the case if the adversary corrupts at most ρ -fraction of the bits communicated, where $\rho \in (0, 1)$ is a fixed small constant defined below.

Good and Bad Rounds

We say that a block is *heavily corrupted* if it contains more than ηL bit corruptions, where $\eta \in (0, 1)$ is the error-resilience of the underlying *inefficient* interactive coding scheme (Theorem 2), and recall that L is the number of bits communicated in an encoded block. If a block is not heavily corrupted then both parties decode it correctly.

We say that a *hash collision* occurred in round t if the transcripts v_A^{t-1} and v_B^{t-1} are distinct and have the same length, but

$$h_{s_P}(v_A^{t-1}) = h_{s_P}(v_B^{t-1}) \quad \text{for at least one } P \in \{A, B\}.$$

This definition refers to the *actual* seeds, not to a corrupted seed appearing in a decoded header.

We say that a round t is bad if it is heavily corrupted or has a hash collision. Otherwise, we say that it is good. We denote by $B_t \in \{0, 1\}$ the event that round t is bad.

The potential function. We use the same potential as in our analysis of Schulman's simulation, except that distances are now measured in blocks:

$$\Phi_t = 2g_t - \max\{\ell_A^t, \ell_B^t\} + 4 \sum_{s=1}^t B_s. \quad (4)$$

In particular, $\Phi_0 = 0$.

The term $2g_t - \max\{\ell_A^t, \ell_B^t\}$ rewards correct common progress and penalizes the longer tentative transcript for being ahead of the common prefix. In a good round, either both parties extend a correct common transcript, or backtracking reduces this penalty.

Claim 3. For every $t \in [N]$, $\Phi_t \geq \Phi_{t-1} + 1$.

Proof. We consider separately the cases $B_t = 0$ and $B_t = 1$.

Case 1: $B_t = 0$. Both parties correctly decode the block, and neither of their true seeds causes a collision between unequal same-length starting transcripts. In particular, both decode the true headers, and their own-header checks pass. There are three possibilities.

- *The transcripts coincide.* By Claim 2, their common transcript is correct. The hash tests pass, and the candidate continuation is the correct next block. Both parties therefore append this block. Hence $g_t = g_{t-1} + 1$, and the maximum depth also increases by 1.
- *The transcripts have different lengths.* The party with the longer transcript deletes one block, and the other party stays. The number of common blocks does not change. Since the two depths differed by at least one block, the maximum depth decreases by exactly 1.
- *The transcripts have the same length but are distinct.* Both hash tests fail, so both parties delete one block. The number of common blocks is shorter than both starting transcript and is not removed. Thus $g_t = g_{t-1}$, while the maximum depth decreases by 1.

In all three possibilities,

$$2g_t - \max\{\ell_A^t, \ell_B^t\} = 2g_{t-1} - \max\{\ell_A^{t-1}, \ell_B^{t-1}\} + 1.$$

Since $B_t = 0$, the cumulative number of bad rounds does not increase.

Case 2: $B_t = 1$. Each party changes its tentative transcript by at most one block. Consequently,

$$g_t \geq g_{t-1} - 1, \quad \max\{\ell_A^t, \ell_B^t\} \leq \max\{\ell_A^{t-1}, \ell_B^{t-1}\} + 1.$$

The first two terms of the potential decrease by at most 3, while the last term increases by 4. Thus $\Phi_t - \Phi_{t-1} \geq 1$ in this case too. $\square$

Since $\max\{\ell_A^N, \ell_B^N\} \geq g_N$, Claim 3 gives

$$g_N \geq \Phi_N - 4 \sum_{t=1}^N B_t \geq N - 4 \sum_{t=1}^N B_t. \quad (5)$$

It remains to bound the number of bad rounds.

Counting Heavily Corrupted Rounds

Let U be the number of heavily corrupted rounds, and let E be the total number of bits corrupted. Since each such round costs more than ηL corruptions,

$$U \leq \frac{E}{\eta L}.$$

Set

$$\rho = \eta/16. \quad (6)$$

The encoded protocol has communication complexity NL . If $E \leq \rho NL$, then

$$U \leq \frac{\rho N}{\eta} = \frac{N}{16}. \quad (7)$$

Counting Hash Collisions

Let F_t indicate a hash collision in round t , and set $F = \sum_{t=1}^N F_t$. Condition on the entire execution before the fresh seeds in round t are sampled. The starting transcripts are now fixed. By Claim 1 and a union bound over the two seeds,

$$p := \Pr[F_t = 1 \mid \text{the preceding execution}] \leq 2 \cdot \frac{k}{Q} \quad (8)$$

Claim 4. $\Pr[F \geq N/16] \leq 2^{-k}$.

Proof. By the union bound, for every $m \in [N]$,

$$\Pr[F \geq m] \leq \binom{N}{m} p^m.$$

Setting $m = N/16$ we conclude that

$$\Pr[F \geq N/16] \leq 2^N \cdot \left(\frac{2k}{Q}\right)^{N/16} \leq 2^{-k},$$

for a large enough $Q = \text{poly}(k)$. $\square$

Putting Everything Together

Every bad round is counted by U or by F , so $\sum_{t=1}^N B_t \leq U + F$. This, together with Claim 4 and Equation (7), implies that

$$\Pr\left[\sum_{t=1}^N B_t \geq N/8\right] \leq 2^{-k}$$

This, together with Equation (5) and (7), implies that

$$g_N \geq N - 4 \sum_{t=1}^N B_t \geq N - 4 \cdot N/8 = N/2.$$

Therefore the two tentative transcripts share at least $\frac{N}{2}b \geq k$ correct bits, and both parties output the original transcript.

References

- [1] Zvika Brakerski and Yael Tauman Kalai. Efficient interactive coding against adversarial noise. In *Proceedings of the 53rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 160–166, 2012.
- [2] Zvika Brakerski, Yael Tauman Kalai, and Moni Naor. Fast interactive coding against adversarial noise. *Journal of the ACM*, 61(6):35:1–35:30, 2014.
- [3] Leonard J. Schulman. Deterministic coding for interactive communication. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 747–756, 1993.
- [4] Leonard J. Schulman. Deterministic coding for interactive communication. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 747–756. ACM, 1993.